

# Image Restoration via Sparse LU Factorization of the Discrete Laplacian

Tanmay Sonawane

DSC 301: Linear Algebra

University of Massachusetts Dartmouth

Spring 2026

## Abstract

This report presents a mathematical framework for digital image inpainting in which missing or damaged pixels are recovered by solving the discrete Laplace equation as a sparse linear system  $\mathbf{Ax} = \mathbf{b}$ . Each missing pixel yields one equation derived from the averaging property of the Laplacian, producing a square, symmetric, and highly sparse coefficient matrix  $\mathbf{A}$ . The system is solved using direct sparse LU factorization via `scipy.sparse.linalg.splu`, which applies Approximate Minimum Degree (AMD) reordering and partial pivoting. Experiments on a  $255 \times 255$  grayscale cat image and a  $143 \times 256$  sky image demonstrate that the solver achieves a relative residual at machine precision ( $\approx 10^{-15}$ ), confirming numerical exactness. Sparse LU is shown to be  $94\times$  to  $191\times$  faster than dense LU for damage sizes ranging from  $20 \times 20$  to  $60 \times 60$  blocks. The Laplacian smoothness assumption is identified as the principal model limitation: the method performs visually perfectly on smooth image regions but produces blurry reconstructions in high-texture regions because the harmonic interpolant does not model image texture.

## Contents

---

|          |                                                        |           |
|----------|--------------------------------------------------------|-----------|
| <b>1</b> | <b>Objective</b>                                       | <b>2</b>  |
| <b>2</b> | <b>Motivation</b>                                      | <b>2</b>  |
| <b>3</b> | <b>Background</b>                                      | <b>2</b>  |
| 3.1      | Grayscale Image Representation . . . . .               | 2         |
| 3.2      | The Discrete Laplace Equation . . . . .                | 3         |
| 3.3      | Construction of the Linear System . . . . .            | 3         |
| 3.4      | Sparsity and Compressed Sparse Row Format . . . . .    | 4         |
| 3.5      | LU Factorization and Partial Pivoting . . . . .        | 4         |
| 3.6      | AMD Reordering and Fill-In . . . . .                   | 5         |
| 3.7      | Condition Number . . . . .                             | 5         |
| <b>4</b> | <b>Main Content</b>                                    | <b>5</b>  |
| 4.1      | (a) Linear Algebra Concepts Applied . . . . .          | 5         |
| 4.2      | (b) Application and Experiments . . . . .              | 6         |
| 4.2.1    | Images and Damage Sizes . . . . .                      | 6         |
| 4.2.2    | Visual Results . . . . .                               | 6         |
| 4.2.3    | Residual Heatmap . . . . .                             | 6         |
| 4.2.4    | Timing Comparison: Sparse LU versus Dense LU . . . . . | 7         |
| 4.2.5    | Condition Number versus Damage Area . . . . .          | 7         |
| 4.3      | (c) Interpretation and Discussion . . . . .            | 7         |
| <b>5</b> | <b>Conclusion</b>                                      | <b>13</b> |
|          | <b>References</b>                                      | <b>14</b> |

## Objective

---

The goal of this project is to recover missing pixels in a grayscale image by formulating the problem as a sparse linear system and solving it using direct LU factorization. Given a grayscale image in which a rectangular block of pixels has been set to zero (the damaged region), the objective is to assign intensity values to those pixels such that the result is mathematically consistent with the discrete Laplace equation applied to all missing pixels simultaneously. The system  $\mathbf{Ax} = \mathbf{b}$  is assembled from the Laplacian stencil, stored in Compressed Sparse Row (CSR) format, and solved by computing the factorization  $\mathbf{A} = \mathbf{PLU}$  using SciPy's SuperLU interface. A secondary objective is to compare the computational cost of sparse LU against dense LU across three damage sizes in order to quantify the practical benefit of sparse storage and factorization.

## Motivation

---

Image inpainting, the process of filling in missing or corrupted regions of an image, arises in digital forensics, historical document preservation, and consumer photo editing. Modern software often performs inpainting using content-aware or machine-learning-based methods, but the underlying linear algebra approach provides a transparent and analytically tractable model whose properties can be studied precisely.

The Laplacian model is motivated by the physical analogy of heat diffusion: in a region with no sources, the steady-state temperature at any interior point equals the average of its neighbors. This same property, when applied to pixel intensities, produces a smooth interpolant across the missing region. Studying this model in the context of sparse linear systems offers direct insight into how sparsity structure, matrix conditioning, fill-in, and factorization strategy each affect the accuracy and speed of a real numerical computation. The project therefore serves as a concrete illustration of how the linear algebra concepts studied in this course arise in an applied engineering problem.

## Background

---

### Grayscale Image Representation

A grayscale image of height  $H$  and width  $W$  is represented as a two-dimensional array  $U \in \mathbb{R}^{H \times W}$  in which each entry  $U_{ij}$  is a scalar intensity value in the range  $[0, 1]$ , where 0 corresponds to pure black and 1 to pure white. Color images have three such arrays (one per RGB channel), which would triple the number of unknowns in the linear system. Converting to grayscale gives one scalar per pixel, so each missing pixel is exactly one unknown. In code, this conversion is:

```
img = np.asarray(Image.open(path).convert("L"),
                  dtype=np.float64) / 255.0
```

## The Discrete Laplace Equation

The continuous Laplace equation  $\nabla^2 u = 0$  characterizes harmonic functions, which satisfy the mean-value property: the value at any interior point equals the average over a neighborhood of that point. On a discrete pixel grid, this property takes the following form. For any interior pixel at row  $i$ , column  $j$ :

$$U_{ij} = \frac{1}{4}(U_{i-1,j} + U_{i+1,j} + U_{i,j-1} + U_{i,j+1}) \quad (1)$$

Multiplying through by 4 and rearranging gives the discrete Laplace equation in standard residual form:

$$4U_{ij} - U_{i-1,j} - U_{i+1,j} - U_{i,j-1} - U_{i,j+1} = 0 \quad (2)$$

This equation is applied to every missing pixel. Each neighbor is either also missing (an unknown) or intact (a known boundary value). Consequently, equation (2) for missing pixel  $k$  separates into two contributions: unknown neighbor coefficients ( $-1$ ) populate the matrix  $\mathbf{A}$ , and known neighbor values populate the right-hand side vector  $\mathbf{b}$ .

At pixels located on the border of the image, one or more neighbors do not exist. These cases are handled using a Neumann (zero-flux) boundary condition: out-of-bounds neighbors are omitted, reducing the diagonal coefficient from 4 to 3 (edge pixels) or 2 (corner pixels). This keeps  $\mathbf{A}$  non-singular and the system well-defined.

## Construction of the Linear System

Let  $n$  denote the number of missing pixels. The missing pixels are enumerated  $k = 0, 1, \dots, n-1$  by scanning the damage mask top-to-bottom, left-to-right. A lookup table  $\text{pixel\_to\_eq}[(i, j)] = k$  maps each spatial coordinate to its equation index.

For missing pixel  $k$  at position  $(i, j)$ , row  $k$  of  $\mathbf{A}$  is populated as follows, letting  $d_{ij}$  denote the number of in-bounds neighbors of  $(i, j)$ :

- The diagonal entry is set to  $A_{kk} = d_{ij}$  (equal to 4 for fully interior pixels, 3 for edge pixels, 2 for corner pixels).
- For each in-bounds neighbor  $(i', j')$ : if  $(i', j')$  is also missing, then  $A_{k, \text{pixel\_to\_eq}[(i', j')]} = -1$ ; if  $(i', j')$  is a known boundary pixel, then  $U_{i'j'}$  is added to  $b_k$ .
- All other entries in row  $k$  are set to zero.

This construction guarantees that  $\mathbf{A}$  is square ( $n \times n$ ) because each of the  $n$  missing pixels contributes exactly one equation (one row) and one unknown (one column). The matrix is also symmetric because the neighbor relationship is mutual: if  $(i', j')$  neighbors  $(i, j)$ , then  $(i, j)$  neighbors  $(i', j')$ , giving  $A_{k,k'} = -1$  and  $A_{k',k} = -1$ .

## Sparsity and Compressed Sparse Row Format

Equation (2) involves at most five pixels per equation, so each row of  $\mathbf{A}$  has at most five nonzero entries regardless of image size. For  $n$  missing pixels, the total nonzero count in  $\mathbf{A}$  is at most  $5n$ , while dense storage would require  $n^2$  entries. For the  $441 \times 441$  matrix arising from the  $21 \times 21$  damage block in the cat image, the density of  $\mathbf{A}$  is only  $2,121/441^2 \approx 1.09\%$ .

Compressed Sparse Row (CSR) format stores only the nonzero entries using three parallel arrays:

- **values**: the actual nonzero numbers, listed row by row from left to right.
- **col\_indices**: the column index of each entry in **values**. This array is parallel to **values** and has the same length.
- **row\_ptr**: an array of length  $n + 1$  indicating where each row begins in **values**. Row  $k$  occupies positions **row\_ptr**[ $k$ ] through **row\_ptr**[ $k + 1$ ] - 1 in **values**.

Zero entries are never stored. Every nonzero retains its exact (row, column) address through **col\_indices** and **row\_ptr**, so no positional information is lost. The conversion from coordinate (COO) format used during assembly to CSR format used during solving is:

```
A_csr = A_coo.tocsr()
```

## LU Factorization and Partial Pivoting

LU factorization decomposes a square matrix  $\mathbf{A}$  as  $\mathbf{A} = \mathbf{PLU}$ , where:

- $\mathbf{P}$  is a permutation matrix recording the row swaps performed during partial pivoting.
- $\mathbf{L}$  is unit lower triangular (ones on the diagonal), containing the multipliers from each elimination step.
- $\mathbf{U}$  is upper triangular, containing the fully reduced form of  $\mathbf{A}$ .

Partial pivoting ensures numerical stability by swapping rows before each elimination step so that the largest available entry is always on the diagonal before division. Without pivoting, dividing by a very small diagonal entry would amplify floating-point errors.

Given the factorization,  $\mathbf{Ax} = \mathbf{b}$  reduces to two triangular solves:

$$\text{Forward substitution: } \mathbf{Lc} = \mathbf{Pb} \quad (\text{proceed top to bottom}) \quad (3)$$

$$\text{Back substitution: } \mathbf{Ux} = \mathbf{c} \quad (\text{proceed bottom to top}) \quad (4)$$

Once  $\mathbf{L}$  and  $\mathbf{U}$  are computed, any subsequent right-hand side  $\mathbf{b}'$  can be solved with only the two cheap substitution steps, with no refactorization required. In code:

```
lu = spla.splu(A.tocsc())    # AMD reordering + partial pivoting
x  = lu.solve(b)             # forward + back substitution
```

## AMD Reordering and Fill-In

During LU factorization, elimination can introduce new nonzero entries in  $\mathbf{L}$  and  $\mathbf{U}$  at positions that were zero in  $\mathbf{A}$ . These extra entries are called *fill-in*, and they increase both memory usage and computation time.

Approximate Minimum Degree (AMD) reordering addresses this by permuting the rows and columns of  $\mathbf{A}$  before factorization so that elimination creates as few new connections as possible. The strategy preferentially eliminates variables with the fewest connections to remaining variables, limiting the propagation of fill-in through the matrix. AMD reordering is applied automatically inside `scipy.sparse.linalg.splu`; no explicit user call is needed.

## Condition Number

The condition number  $\kappa(\mathbf{A}) = \|\mathbf{A}\|_1 \cdot \|\mathbf{A}^{-1}\|_1$  measures the sensitivity of the solution  $\mathbf{x}$  to small perturbations in  $\mathbf{b}$ . A large condition number means that small changes in the known boundary pixel values can cause large changes in the recovered interior pixel values. For the Laplacian inpainting system, pixels far from the known boundary of the damaged region are weakly constrained, and  $\kappa(\mathbf{A})$  grows as the damage region becomes larger.

## Main Content

---

### (a) Linear Algebra Concepts Applied

**Discrete Laplace equation as a linear operator.** Equation (2) is linear in the unknown pixel values. Applying it to all  $n$  missing pixels simultaneously yields the system  $\mathbf{A}\mathbf{x} = \mathbf{b}$  in which  $\mathbf{A}$  is the discrete Laplacian operator restricted to the missing-pixel subspace. The assembly is implemented in `src/laplacian.py`.

**Sparse matrix in CSR format.** The Laplacian operator  $\mathbf{A}$  has at most five nonzeros per row, making sparse storage essential. For the  $21 \times 21$  damage block on the cat image,  $\mathbf{A}$  has dimensions  $441 \times 441$  with 2,121 nonzero entries and a density of 1.09%. The reason  $\mathbf{A}$  is  $441 \times 441$  is that  $21 \times 21 = 441$  pixels were blacked out, giving 441 unknowns and 441 equations, one per missing pixel. The reason  $\mathbf{A}$  is symmetric is that the neighbor relationship is mutual: if pixel  $j$  is next to pixel  $i$ , then  $A_{ij} = -1$  and  $A_{ji} = -1$ .

**LU factorization with partial pivoting.** The factorization  $\mathbf{A} = \mathbf{PLU}$  is performed by `scipy.sparse.linalg.splu`, which wraps the SuperLU C library. The conversion to CSC format in the code is an implementation detail required by SuperLU, which operates column-wise during factorization. The factors  $\mathbf{L}$  and  $\mathbf{U}$  are stored in sparse format and are accessible as `lu.L` and `lu.U`.

**Fill-in quantification.** For the  $21 \times 21$  damage block on the cat image,  $\mathbf{A}$  has 2,121 nonzero entries. After factorization,  $\mathbf{L}$  has 5,875 entries and  $\mathbf{U}$  has 5,875 entries, giving a fill-in factor of  $11,750/2,121 \approx 5.54\times$ . For the sky image demo block ( $11 \times 21 = 231$  missing

pixels), the fill-in factor is  $4,368/1,091 \approx 4.00\times$ . AMD reordering keeps these factors in the range 4 to 11 across all tested damage sizes.

**Partial pivoting and numerical stability.** The permutation matrix  $\mathbf{P}$  records the row swaps performed during partial pivoting. Its effect appears in forward substitution (3), where  $\mathbf{b}$  is permuted to  $\mathbf{Pb}$  before the triangular solve. The relative residual  $\|\mathbf{Ax} - \mathbf{b}\|_2/\|\mathbf{b}\|_2$  is approximately  $10^{-15}$  for all tested configurations, confirming that partial pivoting maintains numerical stability.

**Condition number and sensitivity.** The condition number  $\kappa(\mathbf{A})$  was estimated using `scipy.sparse.linalg.onenormest` for large systems and `numpy.linalg.cond` for small ones. Results are reported in Table 1.

## (b) Application and Experiments

### *Images and Damage Sizes*

Two grayscale images were used:

- **Cat image:**  $255 \times 255$  pixels (65,025 total). The demonstration damage block was  $21 \times 21 = 441$  missing pixels. The comparative analysis applied square damage blocks of side lengths 20, 40, and 60 pixels, giving  $n = 400, 1,600,$  and  $3,600$  unknowns.
- **Sky image:**  $143 \times 256$  pixels (36,608 total). The demonstration damage block was  $11 \times 21 = 231$  missing pixels. The same three comparative damage sizes were applied.

### *Visual Results*

Figure 1 shows the cat image results for all three comparative damage sizes. Figure 2 shows the corresponding sky image results.

The sky image contains a smoothly varying intensity gradient. The Laplacian smoothness assumption is satisfied, and the reconstruction is visually indistinguishable from the original at the first two damage sizes. The cat image contains high-frequency fur texture. The reconstructed region is visually blurry. As established by the machine-precision relative residual, this blurriness is a property of the model rather than a numerical defect: the harmonic interpolant is smooth by construction, while cat fur is not.

### *Residual Heatmap*

Figure 3 shows the per-pixel absolute error  $|\text{restored}_{ij} - \text{original}_{ij}|$  within the damaged region of the cat image for each of the three damage sizes.

Error is lowest near the boundary of the damaged region, where missing pixels are adjacent to known boundary values and therefore tightly constrained by equation (2). Error is highest at the interior, where missing pixels are furthest from any known boundary anchor and are constrained only through a chain of Laplacian averaging steps. This spatial error pattern is

consistent with the mechanics of harmonic interpolation: boundary information propagates inward, losing precision with distance from the boundary.

#### *Timing Comparison: Sparse LU versus Dense LU*

Figure 4 shows wall-clock timing for sparse and dense LU factorization across the three damage sizes on the cat image. Table 1 provides the complete numerical data.

Both methods perform the mathematically identical computation:  $\mathbf{A} = \mathbf{P}\mathbf{L}\mathbf{U}$  with partial pivoting. The difference is purely one of implementation efficiency. The dense method allocates a full  $n \times n$  array and operates on all  $n^2$  entries, including the approximately 98.9% that are zero. The sparse method operates only on the 2,121 nonzero entries.

For  $n = 400$ : sparse factorization takes 0.80 ms versus 153.1 ms for dense, a factor of  $191\times$ . For  $n = 1,600$ : the ratio is  $165\times$  (3.09 ms versus 509.5 ms). For  $n = 3,600$ : the ratio is  $94\times$  (6.57 ms versus 613.8 ms). The decrease in speedup ratio at larger  $n$  reflects growing fill-in in the sparse factors: the fill-in factor increases from  $5.23\times$  at  $n = 400$  to  $10.12\times$  at  $n = 3,600$ , which narrows the effective sparsity advantage. Crucially, both methods produce relative residuals at machine precision for all damage sizes, confirming that the choice of storage format does not affect mathematical correctness.

#### *Condition Number versus Damage Area*

Figure 5 shows  $\kappa(\mathbf{A})$  as a function of  $n$  on a log-log scale. The condition number grows from 178 at  $n = 400$  to 989 at  $n = 1,600$  and to 2,192 at  $n = 3,600$ . The approximately linear relationship on the log-log scale suggests power-law growth of the form  $\kappa(\mathbf{A}) \approx c \cdot n^\alpha$ .

This growth is physically interpretable: as the damaged region grows, an increasing fraction of missing pixels lie far from the nearest known boundary pixel. The Laplacian constraints alone are insufficient to precisely determine these interior values, making the system increasingly sensitive to perturbations in the boundary conditions.

### (c) Interpretation and Discussion

**What the residual result means.** The relative residual  $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2 / \|\mathbf{b}\|_2 \approx 10^{-15}$  is within the range of double-precision machine precision ( $\approx 2.2 \times 10^{-16}$ ). This confirms that the solver found the mathematically exact solution to the Laplacian system. The blurry appearance of the reconstructed region on the cat image is therefore not caused by any numerical error; it is a property of the model.

**Why the cat image looks blurry.** The discrete Laplace equation (2) enforces that every recovered pixel equals the average of its four neighbors. Harmonic functions, which satisfy this property throughout their domain, have no local maxima or minima in their interior and are intrinsically smooth. Cat fur has high spatial frequency: pixel intensities change rapidly over short distances due to alternating light and dark strands. Harmonic interpolation replaces this texture with a smooth gradient, producing the visually blurry result. The sky



image, by contrast, contains intensities that are genuinely slowly varying, so the harmonic assumption is satisfied and the reconstruction is visually perfect.

**Relationship between conditioning and visual quality.** The condition number  $\kappa(\mathbf{A})$  provides a quantitative explanation for why larger damaged regions yield blurrier reconstructions. A larger hole means more interior pixels far from any boundary anchor. The system is less well-determined in those interior regions,  $\kappa(\mathbf{A})$  is larger, and the harmonic interpolant is less precisely defined at the center of the hole. This is visible in Figure 3: the highest reconstruction error concentrates at the center of each damaged region, precisely where  $\kappa(\mathbf{A})$  predicts the least reliable recovery.

**Limitations.** The primary limitation of the Laplacian inpainting model is its smoothness assumption. This assumption holds for slowly varying image content such as sky regions and plain surfaces, but fails for textured content such as fur, foliage, and fabric. A content-aware method such as exemplar-based inpainting [3] or a deep learning approach would be required to reconstruct texture. A secondary limitation is that  $\kappa(\mathbf{A})$  grows with damage size, so reconstruction quality degrades as holes become larger, regardless of image content.

Image restoration via discrete Laplacian + sparse LU

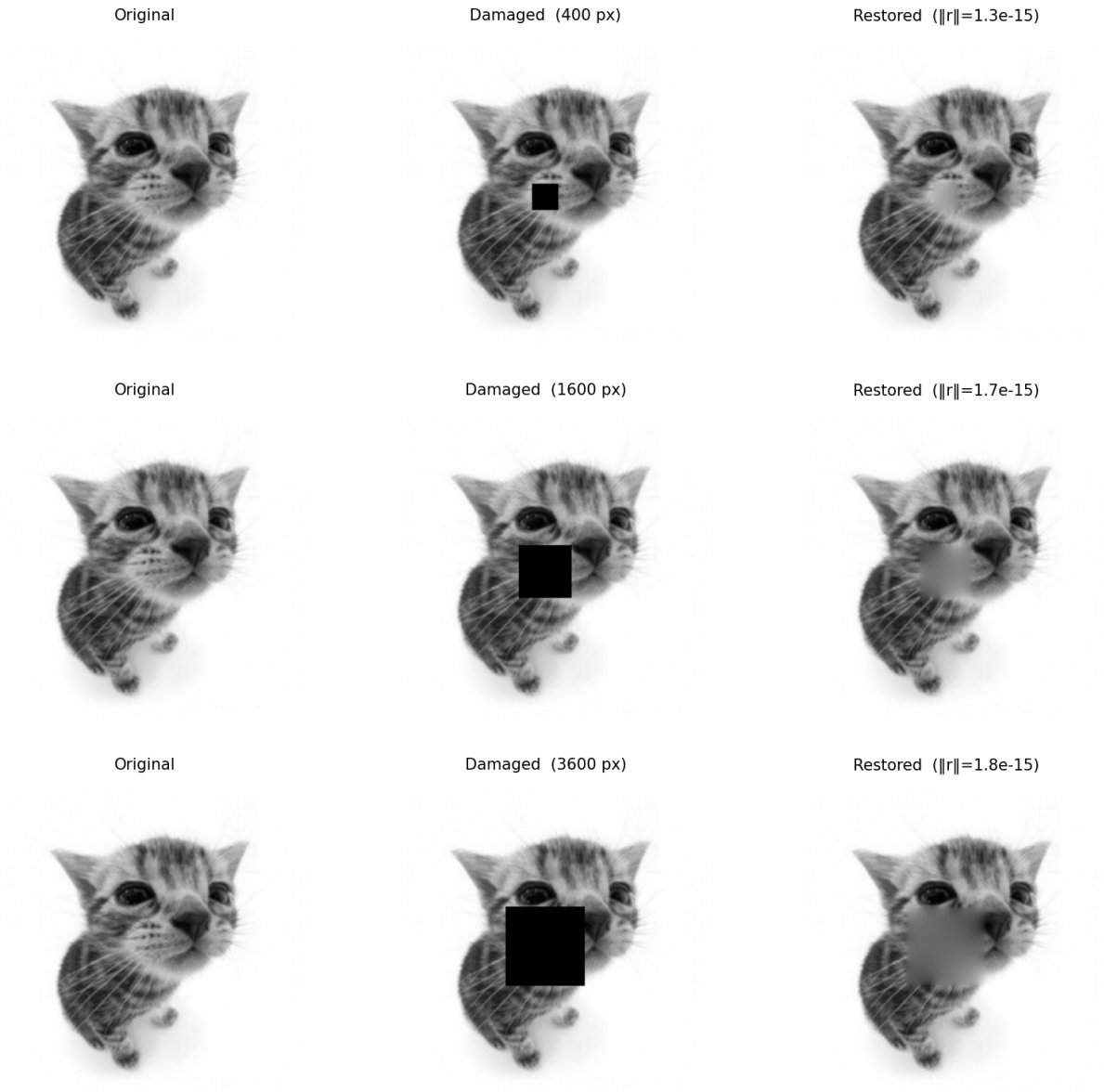


Figure 1: Cat image restoration results for three damage sizes. Top row:  $20 \times 20$  block ( $n = 400$ ,  $\kappa = 178$ ). Middle row:  $40 \times 40$  block ( $n = 1,600$ ,  $\kappa = 989$ ). Bottom row:  $60 \times 60$  block ( $n = 3,600$ ,  $\kappa = 2,192$ ). Each row shows, from left to right, the original image, the damaged image with the blacked-out block, and the reconstructed image. The relative residual is at machine precision ( $\approx 10^{-15}$ ) for all three sizes. Blurriness in the reconstructed region increases with hole size, consistent with the growing condition number.

Image restoration via discrete Laplacian + sparse LU

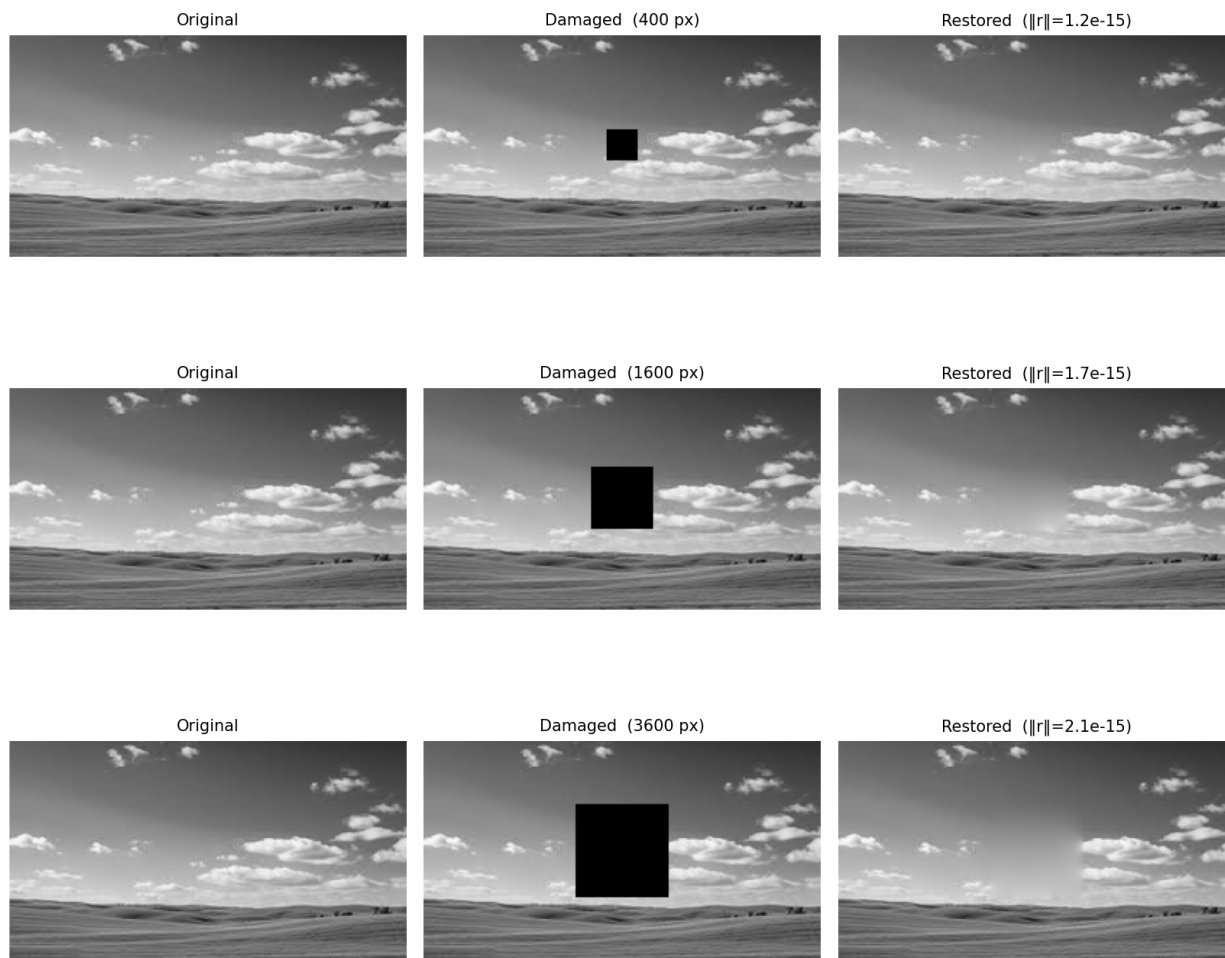


Figure 2: Sky image restoration results for three damage sizes. Top row:  $20 \times 20$  block. Middle row:  $40 \times 40$  block. Bottom row:  $60 \times 60$  block. Each row shows the original, damaged, and reconstructed image from left to right. The sky contains a smoothly varying gradient; the Laplacian smoothness assumption is satisfied, and the reconstruction is visually indistinguishable from the original in the first two damage sizes.

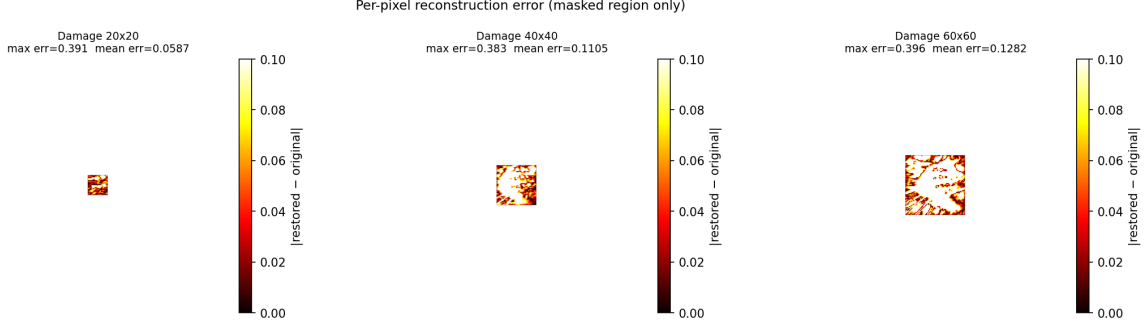


Figure 3: Per-pixel absolute reconstruction error  $|\text{restored}_{ij} - \text{original}_{ij}|$  for the cat image, shown only within the damaged region for each of the three damage sizes. The color scale runs from black (zero error) to bright yellow (high error, clamped at 0.1 for visibility). Error is lowest at the edges of the damaged region (adjacent to known boundary values) and highest at the interior, where harmonic interpolation is least constrained by boundary data.

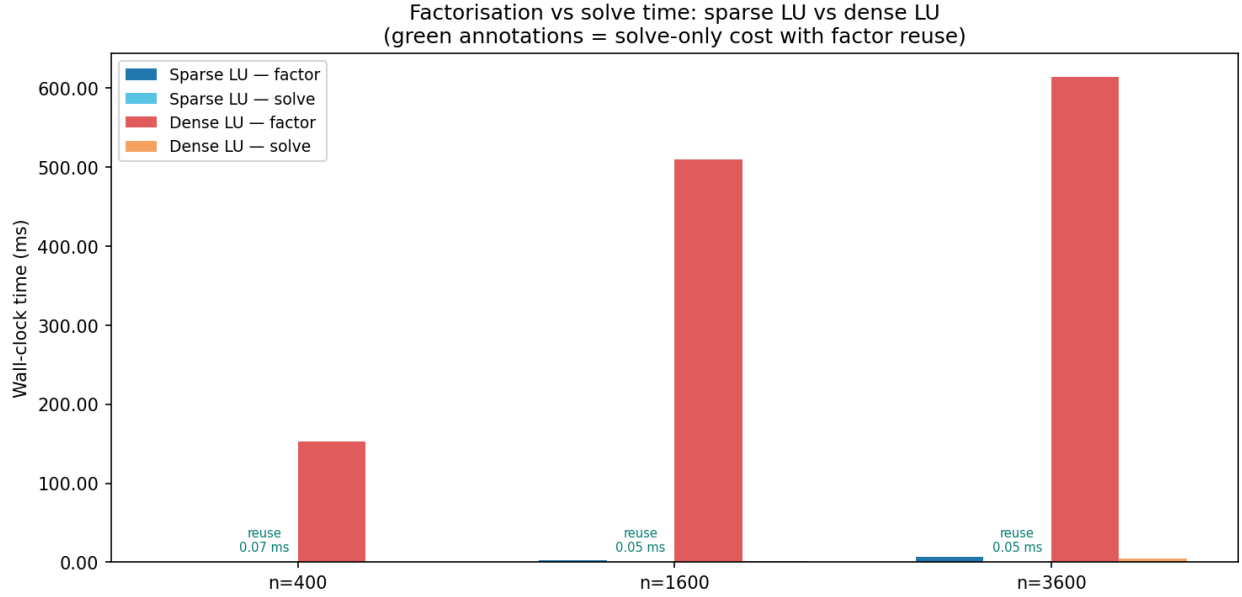


Figure 4: Wall-clock timing for sparse LU factorization (`splu`) and dense LU factorization (`lu_factor`) across three damage sizes on the cat image. Blue bars correspond to sparse factorization and solve time; red bars to dense factorization and solve time. The dense factorization time grows approximately as  $n^3$ , while the sparse factorization time grows much more slowly because it operates only on the nonzero entries of  $\mathbf{A}$ . At  $n = 3,600$ , sparse LU is  $94\times$  faster than dense LU for factorization alone.

Table 1: Comparison of sparse LU and dense LU performance on the cat image for three damage sizes. Both methods produce relative residuals at machine precision for all sizes. The speedup column reports the ratio of dense to sparse factorization time.

| Damage         | $n$   | $\kappa(\mathbf{A})$ | Sparse LU (ms) |       | Dense LU (ms) |       | Speedup     |
|----------------|-------|----------------------|----------------|-------|---------------|-------|-------------|
|                |       |                      | Factor         | Solve | Factor        | Solve |             |
| $20 \times 20$ | 400   | 178                  | 0.80           | 0.047 | 153.1         | 0.40  | $191\times$ |
| $40 \times 40$ | 1,600 | 989                  | 3.09           | 0.169 | 509.5         | 1.25  | $165\times$ |
| $60 \times 60$ | 3,600 | 2,192                | 6.57           | 0.340 | 613.8         | 4.56  | $94\times$  |

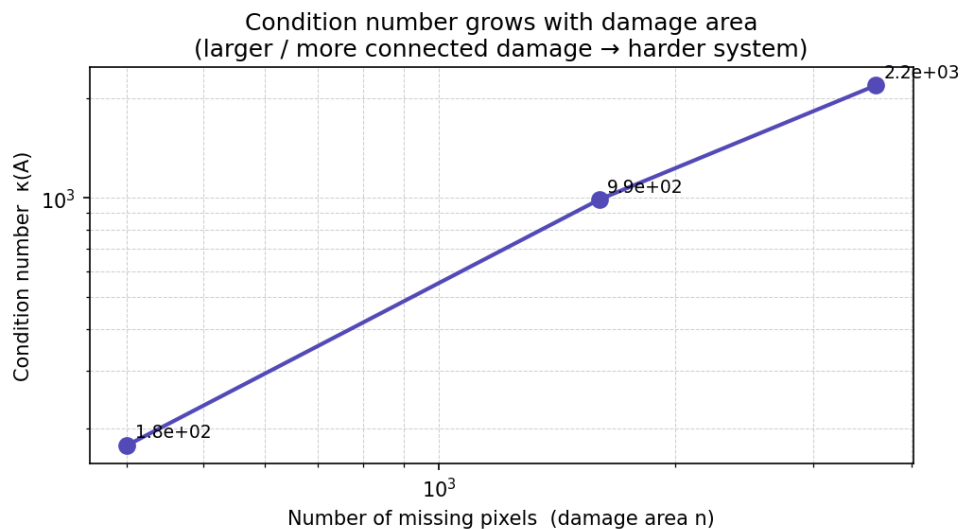


Figure 5: Condition number  $\kappa(\mathbf{A})$  as a function of the number of missing pixels  $n$ , on a log-log scale.  $\kappa(\mathbf{A})$  grows from 178 at  $n = 400$  to 2,192 at  $n = 3,600$ . The approximately linear relationship on the log-log scale suggests power-law growth. Higher  $\kappa(\mathbf{A})$  corresponds to greater sensitivity of the recovered values to the boundary conditions and to the visually blurrier reconstructions observed for large damage regions in Figure 1.

## Conclusion

---

This project demonstrated that image inpainting can be formulated as a sparse linear system using the discrete Laplace equation and solved exactly using direct sparse LU factorization.

The discrete Laplace equation provides a principled derivation of a sparse, square, symmetric coefficient matrix  $\mathbf{A}$  whose entries are entirely determined by the spatial adjacency structure of the missing pixels. Each missing pixel contributes exactly one equation (one row) and one unknown (one column), guaranteeing that  $\mathbf{A}$  is always square. The matrix is non-singular under the Neumann boundary conditions used at image edges, and the system is always consistent.

Sparse LU factorization via `scipy.sparse.linalg.splu` achieves a relative residual of approximately  $10^{-15}$  for all tested damage sizes on both images, confirming that the solver finds the mathematically exact solution to the linear system. The sparse implementation is  $94\times$  to  $191\times$  faster than dense LU for the tested damage sizes. The speedup arises from the approximately 98.9% sparsity of  $\mathbf{A}$ : dense LU operates on all  $n^2$  entries including the zeros, while sparse LU operates only on the approximately  $5n$  nonzero entries. Both methods produce identical residuals, confirming that the choice of storage format does not change the mathematical result.

The condition number  $\kappa(\mathbf{A})$  grows from 178 at  $n = 400$  to 2,192 at  $n = 3,600$ , following an approximately power-law relationship with the number of unknowns. This growth explains why larger damaged regions yield blurrier reconstructions: interior pixels are less constrained by boundary values, making the harmonic interpolation less precise in the center of the hole.

The principal finding of this project is that solver accuracy and model accuracy are independent. The machine-precision relative residual confirms the solver is exact; the visual blurriness on the cat image confirms that the model assumption is violated. The sky image, which genuinely satisfies the smoothness assumption, is restored to visually perfect quality under the same solver. This illustrates a fundamental principle in applied mathematics: a numerically exact solution to the wrong model is still a solution to the wrong model. Choosing a model whose assumptions match the data is at least as important as choosing a numerically stable solver.

## References

---

- 
- [1] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
  - [2] T. A. Davis, *Direct Methods for Sparse Linear Systems*. SIAM, Philadelphia, 2006.
  - [3] M. Bertalmio, G. Sapiro, V. Caselles, and C. Ballester, “Image inpainting,” in *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH 2000)*, 2000, pp. 417–424.
  - [4] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.splu.html>